LoomShield — Round 2 Project Document

Completed features, technology, changes to the plan and Round 3 improvements

PROJECT LINKS

Working project: https://inkloom-artloomshield-frontend.vercel.app/
Source code: https://github.com/isaacincognito007-cloud/Inkloom-artLoomShield
Hosted backend (API): https://inkloom-artloomshield.onrender.com

The public deployment exists so evaluators can try LoomShield. It runs as a clearly labelled demo workspace, and the backend sits on Render's free tier, so the first request after a quiet period can be slow while it wakes up. The README explains this, and the setup wizard has built-in fallbacks so it stays usable in the meantime.

WHAT LOOMSHIELD DOES

LoomShield checks content against a brand's guidelines (tone and rules, preferred words, approved facts, colours and logo, and accessibility), flags what needs a human look, and keeps an audit history of review and approval decisions. Its analysis runs on engines built into the backend, so no API key or third-party AI service is required.

1. COMPLETED FEATURES

Guided "New Audit" setup `commit 823c348`

- Interactive wizard with a progress bar, step counter, large selectable tiles, a review summary and an add-content step.
- 22 validated rule-set fragments covering 6 industries, 6 occasions, 6 locales and 4 tones, merged from the user's answers into a tailored brand profile.
- Tailored sample text that produces at least three intentional findings, so a new user sees an audit work straight away.
- Every setup creates its own isolated brand profile (capped at 25 in demo mode) and never alters the seeded demo brands.
- Role-checked, rate-limited endpoints: `GET /api/wizard/options`, `POST /api/wizard/preview` and `POST /api/wizard/profile`.
- Rule patterns are checked for regular-expression (ReDoS) safety at startup, and 9 automated tests cover the wizard.

SaaS-grade polish `commit 46c9dfa`

- CSS design tokens: a slate neutral scale, one primary action colour, state colours specified for WCAG AA contrast, a 4–32 px spacing scale, consistent corner radii, sticky table headers, skeleton loaders and high-visibility keyboard focus rings.
- Redesigned Overview: a welcome header, three KPI cards (documents audited, open findings, approved content) scoped to the active brand and version, and a prominent "Start a new audit" card.
- Plain-language labels across navigation, tabs and status badges, driven by one shared labels map, for example "Brand guidelines", "Preferred words", "Approved facts", "Couldn't check" and "Needs a human look".

Clarity for first-time users and evaluators `commit 1a29a71`

- A "Start here" walkthrough on the Overview in demo mode: answer a few questions, try the example, then review issues and approve.
- An About & Roadmap view covering what LoomShield does across its six audit pillars, which Round 2 capabilities are backed by code and automated tests, and what is planned for Round 3.
- A README quickstart ("Try it in 60 seconds") with public URLs, the free-hosting wake-up notice and the Render deployment configuration.

Setup chip and "Redo setup" `commit 4cb7876`

- A "Set up for: …" badge on the Brand guidelines header shows the answers the active profile was built from, with a "Redo setup" button beside it that reopens the wizard at step 1.

Checked on the live deployment at the Slice 4 release

- The frontend returned HTTP 200 and the new elements (Start-here card, setup chip, wizard stage) were present in the live page.
- The session endpoint reported a demo workspace on the hosted deployment, and the engine-status endpoint reported both analysis engines' self-tests as passing.

2. TECHNOLOGY AND IMPLEMENTATION DETAILS

Frontend: plain HTML, CSS and JavaScript (`index.html` , `app.js` , `styles.css`) on Vercel. CSS custom properties carry the design tokens, one labels map keeps wording consistent, and the wizard's questions and options also exist client-side as a fallback.

Backend: a Python FastAPI API under `/api/` on Render. `backend/wizard.py` handles startup validation, the ReDoS-safety check, a pure `compose()` function that merges answers, sample-text generation and atomic profile creation; rule-set fragments live in `backend/rulesets/` .

Analysis engines: Tier 1 is a deterministic rules-and-patterns engine; Tier 2 is an embedded scikit-learn TF-IDF NLP engine. Both run inside the backend, need no API keys, and report self-test results at `/api/engine-status` .

Data: audit results, revisions, approvals and audit events are stored in an SQLite database by the backend (`audit_events` , `revisions` and `approvals` tables).

Safeguards: role-checked and rate-limited wizard endpoints, a 5 MB upload limit enforced in both the interface and the backend, and a demo-mode cap on generated profiles.

Testing: 9 automated wizard tests in `tests/test_wizard.py` .

Hosting: Vercel for the frontend and a Render free instance for the backend, with the Render URL configured in the Vercel project.

3. CHANGES TO THE ORIGINAL PLAN

Presets create a new profile instead of switching an existing brand

The goal is presets that switch all five Brand Vault sections together. In Round 2, the guided setup builds a new, isolated profile from validated rule-set fragments, so a setup cannot overwrite existing custom guidelines. Applying a preset to an existing brand, with rollback, moves to Round 3.

Upload size

The target is 100 MB per file. Round 2 ships with a 5 MB cap enforced in both the interface and the backend; chunked streaming for files up to 100 MB is scheduled for Round 3.

Scanned PDFs

PDFs without selectable text cannot be read yet because there is no OCR. They are flagged as not checkable, with guidance to supply selectable text or run OCR first.

Added scope for clarity

The guided wizard, plain-language labels, evaluator walkthrough and About & Roadmap view were added so first-time users and evaluators reach a first result quickly.

Demo deployment, not a production service

The public deployment is a labelled demo workspace built for evaluators. Roles are previews in demo mode; real authentication is part of the production roadmap.

Not claimed in this submission

Saved data surviving redeploys, per-visitor isolation of demo data, a formal accessibility audit (Lighthouse/axe) and a full traceability check of every Round 1 promise are not claimed here; they remain on the roadmap.

4. THREE MAJOR IMPROVEMENTS FOR ROUND 3

Each improvement will ship with a specification, milestones, acceptance criteria and a short demo script.

1. Clarity Rewriter

Vague guideline wording causes inconsistent decisions and repeated overrides. The Clarity Rewriter scores each rule for ambiguity using explainable inputs (documented vagueness indicators, missing measurable criteria, override and dismissal rates above a documented minimum sample, and previously accepted corrections), then proposes a clearer rewrite grounded in verified facts and accepted rule changes. A Brand Administrator must approve it, which creates a new Brand Vault version; we then measure whether violations and overrides change afterwards. It builds on the existing Guideline Ambiguity Radar; the new parts are rewrite proposals, administrator approval with Vault versioning, and before-and-after measurement.

Done when: an administrator can approve a rewrite, a new Vault version appears, and the app reports violations and overrides for that rule before and after the change.

2. Visual heatmap and clear-space check

Shows where a banner or image breaks the brand's visual rules. The logo is located by one of three supported methods: a user-drawn bounding box, a named SVG group, or a template match against an approved logo asset. Clear space is configured as a multiple of the logo's height and shown on a configurable grid heatmap. Contrast checks cover only regions the application can genuinely inspect; without OCR, it does not claim to read text in arbitrary images. Labelled banner fixtures with known violations make the feature testable.

Done when: every labelled fixture produces its expected outcome, and any misses are reported openly.

3. Signed compliance certificate

Gives each approved item a tamper-evident certificate. Its JSON is canonicalised and signed with a documented signature algorithm; the private key is kept in a secret store and the public verification key is published with a key ID. The certificate records the content hash, Vault version, reviewer and approver identities, signature, timestamp, verification URL and revocation status. Where stronger tamper evidence is needed, the ledger head or timestamp is anchored outside the application database. We call it tamper-evident and make no claim of legal validity.

Done when: a certificate verifies against the public key, and changing any field makes verification fail.